Threat Model and Secure Configuration

Student Login Portal | Beginner Defensive Security Project

Purpose

Identify realistic threats to a small web application, prioritize risk, and recommend proportionate defensive controls. This report is a design exercise; it does not claim that a real application was tested.

Scope

A fictional student portal that supports account login, profile viewing and profile updates. Components include a browser, web application, authentication service, database, audit logging, and an optional external email/MFA provider.

Deliverables

1. Threat-model diagram
2. STRIDE threat analysis and risk register
3. Prioritized security hardening checklist

Prepared as a learning artifact. Review and adapt the assumptions before submission.

1. Application Overview

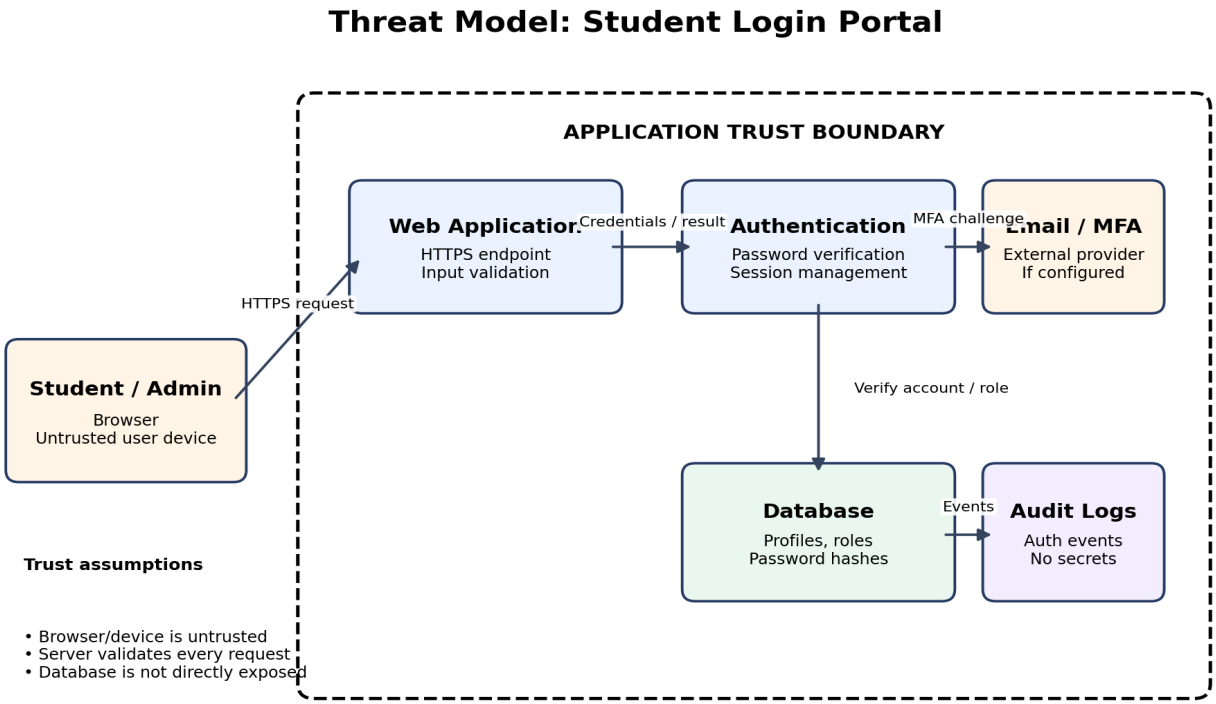
The Student Login Portal allows students and administrators to authenticate and access role-appropriate information. The browser is treated as untrusted. All validation and authorization decisions must be enforced on the server. The database must not be directly reachable by users.

Assets to protect

- Account credentials and password hashes
- Student profile and other personal information
- Session identifiers and authentication state
- Administrator privileges and protected functions
- Application availability
- Security and audit logs

2. Threat-Model Diagram

The dashed box marks the application trust boundary. The external user/device is outside it. Email/MFA is shown as an external dependency if enabled. Arrows represent logical data flows.



Key assumptions: use HTTPS; validate requests server-side; keep database access private; store passwords using a modern password-hashing function; do not log secrets.

3. STRIDE Threat Analysis

Category	Example abuse case	Recommended controls
Spoofing	An attacker uses a stolen or guessed student password to impersonate a student.	Strong password hashing; MFA for sensitive accounts; rate limiting; secure recovery.
Tampering	A student changes a profile identifier in a request to edit another student's record.	Server-side object-level authorization; validate ownership on every request.
Repudiation	A user denies changing an account email or role.	Timestamped audit events; record actor, action, target and outcome; protect logs.
Information disclosure	An endpoint returns another student's profile or exposes secrets in errors/logs.	Least-privilege access; data minimization; safe errors; secret-free logs; encryption in transit.
Denial of service	Repeated login or resource-heavy requests make the portal unavailable.	Per-account and per-IP rate limits; request/time limits; monitoring; upstream protections.
Elevation of privilege	A student invokes an administrator endpoint directly.	Deny by default; server-side role checks; least privilege; tests for access-control boundaries.

4. Risk Register

Scoring method: Likelihood (1–5) × Impact (1–5). These are initial illustrative estimates for a fictional design, not measured findings. Reassess them when the architecture and controls are known.

ID	Risk	L	I	Score	Priority	Rationale / treatment
R1	Unauthorized access to another student's profile	4	5	20	Critical	Potential privacy breach; enforce object-level authorization and test cross-account access.
R2	Student reaches administrator functions	3	5	15	High	Could expose or alter many accounts; deny by default and verify roles server-side.
R3	Credential guessing or password reuse compromises an account	4	4	16	Critical	Use password hashing, rate limiting, MFA where appropriate, and safe recovery.
R4	Sensitive data appears in logs or error messages	3	4	12	High	Use safe error responses, redact secrets, and restrict log access.
R5	Repeated requests degrade login availability	3	4	12	High	Apply rate limits, request limits, alerting and operational monitoring.
R6	Important account changes cannot be investigated	2	3	6	Medium	Log relevant security events with timestamps and outcomes; protect log integrity.

Priority bands used: 1–4 Low; 5–9 Medium; 10–15 High; 16–25 Critical. The numeric score helps comparison; context and exposure should also influence final decisions.

5. Prioritized Security Hardening Checklist

Priority 1 — address first

■ Enforce server-side authorization on every protected operation, including checks that a student owns the requested record.
■ Restrict administrator endpoints using server-side role checks and deny-by-default rules.
■ Store passwords using a modern password-hashing algorithm (for example, Argon2id) with appropriate parameters.
■ Use HTTPS and secure session handling; protect session cookies with Secure, HttpOnly and an appropriate SameSite setting; rotate session IDs after authentication.

Priority 2 — reduce common abuse

■ Rate-limit authentication attempts and consider both account- and source-based signals.
■ Use generic login and recovery responses that do not reveal whether an account exists.
■ Validate input server-side and use parameterized database queries.
■ Expire sessions appropriately and invalidate them on logout; require re-authentication for sensitive changes where suitable.

Priority 3 — improve detection and resilience

■ Log successful and failed authentication, authorization failures, and important account changes.
■ Exclude passwords, session tokens, API keys and other secrets from logs.
■ Restrict database and log access using least privilege; protect backups and sensitive data.
■ Monitor unusual login patterns and configure alerting and request/resource limits.

6. Verification Plan

Run these checks only in an application you own or have permission to test. Record the date, test method, expected result, actual result and evidence. Do not claim a test passed unless you ran it.

• Authorization: sign in as Student A and attempt to access Student B's record; expected result is denial without disclosing B's data.
• Privilege boundaries: sign in as a student and request an administrator-only function; expected result is denial.
• Rate limiting: make repeated test login attempts in a local test environment; expected result is throttling or temporary protection without locking out legitimate users unfairly.
• Session handling: verify logout invalidates the session and cookie flags are appropriate.
• Logging: trigger a test login failure and verify logs contain useful event details but no password or session token.

7. Conclusion

The initial priorities are access control, administrator privilege boundaries, and account protection because compromise could expose or alter student information. Secure session handling, rate limiting, safe logging, and monitoring provide additional layers of defense. This report is a threat-modeling exercise; implementation and testing evidence must be added separately if those activities are performed.

References for further study

• OWASP Threat Modeling: https://owasp.org/www-community/Threat_Modeling
• OWASP Application Security Verification Standard: https://owasp.org/www-project-application-security-verification-standard/
• OWASP Password Storage Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html